

LABORATORY 2

Dart & Null Safety

classes, null-safe types and collections

What you'll build today



Null safety in code

Reach for `?`, `?.`, `??`, `??=` and late deliberately instead of by trial and error



Classes and inheritance

Model a small domain with an abstract base class and two subclasses



Null-safe collections

Search, filter and default over `List<T>` without ever returning null



Lecture 2, applied

This lab puts Lecture 2 to work: today is the language, not the framework

Where your code goes

You are not starting a new project.

Open the default Flutter project you created and ran in Lab 1, the one already pushed to your GitHub repository.

Create a branch named lab2 before you write anything.

All three tasks are plain Dart classes. Give each one its own file under lib/models/.

Only main.dart holds widget code, just enough to display what your classes produce.

```
lib/  
  main.dart  
  models/  
    user.dart  
    bank_account.dart  
    library_item.dart  
    book.dart  
    magazine.dart  
    library.dart
```

One class per file, lower_snake_case filenames. That is the Dart convention, and the graders read your repository.

The null-safety toolkit

Types and operators

<code>String?</code>	a value that may be null; plain <code>String</code> never can
<code>?.</code>	read a member only if the receiver is non-null
<code>??</code>	use the right-hand value when the left one is null
<code>??=</code>	assign only if the variable is currently null
<code>!</code>	assert non-null; throws at runtime if you are wrong

Declarations and promotion

<code>late</code>	assigned after construction, before its first read
<code>required</code>	a named parameter the caller must pass
<code>= 0.0</code>	default value for an optional parameter
<code>is Book</code>	type test that promotes the variable in the branch
<code>x != null</code>	flow analysis promotes <code>x</code> to non-nullable here

Use ! last, not first. Every `!` you write is a promise the compiler can no longer check for you.

The same rules, in one file

```
class Room {  
    final String code;           // always present  
    final int? seats;           // may be unknown  
    String? projector;          // may be absent  
    late final DateTime opened;  // set later, read after  
  
    Room({required this.code, this.seats, this.projector});  
  
    String describeSeats() =>  
        seats == null ? 'Capacity unknown' : '$seats seats';  
}
```

Read it line by line

code can never be null, so no check is ever needed.

seats and projector may be null, so the compiler forces you to say what happens then.

opened has no value yet; reading it too early throws `LateInitializationError`.

Inside the `== null` branch the compiler knows seats is an `int`, so `$seats` compiles.

r.projector ??= 'none'; then `r.projector?.length` is 4, with no null check written by hand.

TASK I

Users

Work in `lib/models/user.dart`, and show the result from `lib/main.dart`.

1. Create a `User` class with a `String` `name`, an `int?` `age` and a `String?` `email`.
2. Make `name` a required named parameter; leave `age` and `email` optional.
3. Create at least three `User` instances covering different combinations of null and non-null values.
4. Display each user's information safely, showing “Not provided” wherever a value is null.
5. Implement a method `getDisplayAge()` that returns “Age not specified” when `age` is null, and the `age` as a string otherwise.

Hints

`int?` / `String?` for nullable fields

`??` supplies the “Not provided” fallback

`?.` reads through a nullable safely

required vs optional named parameters

if (`age != null`) promotes `age` to `int`

Done when

Three users render, one with every optional field null

`getDisplayAge()` covers both branches

No null-check crash on any of them

Bank accounts

lib/models/bank_account.dart. This task is about late: a field that is not ready at construction time.

1. Create a BankAccount class with a late String accountNumber, assigned after the object is created.
2. Add a required String holderName named parameter to the constructor.
3. Add a double balance property with a default value of 0.0.
4. Add an optional String? bankBranch property.
5. Add a method initializeAccount(String number) that sets the account number.
6. Add a way for callers to tell whether the account number has been set yet.
7. Read the account number through that check, so an uninitialized account shows a message instead of throwing.
8. Create several accounts and display them in a ListView.

Hints

late String: no value at construction

LateInitializationError = read too early

A bool flag, or a String? backing field

double balance = 0.0 in the constructor

ListView.builder, on the next slide

Done when

Every account renders, branch or “Not provided”

An uninitialized account shows a message, not a crash

Just enough UI to see your objects

```
// inside the body: of your Scaffold
ListView.builder(
  itemCount: accounts.length,
  itemBuilder: (context, i) {
    final a = accounts[i];
    return ListTile(
      title: Text(a.holderName),
      subtitle: Text(a.bankBranch ?? 'Not provided'),
      trailing: Text(a.balance.toStringAsFixed(2)),
    );
  },
)
```

You have not had the widgets lecture yet

Widgets are Lecture 3 and Lab 3; today you only need enough of them to see your data.

ListView.builder builds only the rows that are on screen.

?? inside the widget tree is how the “Not provided” rule reaches the display.

The default project already gives you MaterialApp and Scaffold, so put this in body:.

Library management system: the model

The biggest task of the lab. Build the class hierarchy first; the Library itself is on the next slide.

1. Create an abstract class `LibraryItem` with a `String` `title`, a `String?` `author` and a `DateTime?` `publishDate`.
2. Create a `Book` class that extends `LibraryItem` and adds an `int?` `pageCount` and a `String?` `isbn`.
3. Create a `Magazine` class that extends `LibraryItem` and adds an `int?` `issueNumber` and a `String?` `publisher`.
4. Pass the shared fields up to `LibraryItem` through a super constructor in both subclasses.
5. Add a `User?` `borrower` and a `DateTime?` `dueDate` to `LibraryItem`, so any item can be on loan.
6. Reuse the `User` class from Task I as the borrower type; do not write a second one.

Hints

abstract class: cannot be instantiated

class `Book` extends `LibraryItem`

super constructor: `: super(title: title)`

`@override` on anything you replace

`DateTime?`: null until published

Done when

`Book` and `Magazine` both compile with no unset non-null field

Every unknown value is typed with `?`, not defaulted to a dummy

The Library API and borrowing

lib/models/library.dart. A Library class managing the collection with null-safe operations.

1. Give Library a List<LibraryItem> and an addItem(LibraryItem? item) that validates before adding.
2. Implement searchByTitle(String? query), handling a null or empty search term.
3. Implement getItemsByAuthor(String? author), returning the items written by that author.
4. Implement getRecentItems(int? count), falling back to a default count when count is null.
5. Implement borrowing: set borrower and dueDate when an item is taken out.
6. Refuse to lend an item that already has a borrower, and clear both fields when it is returned.

Hints

where(...).toList() to filter

?? supplies the default count

sort by publishDate, nulls last

item.borrower == null means available

Return an empty list, never null

Done when

Every method survives a null argument

Borrowing the same item twice is rejected

A null query returns a sensible list, not an exception

Errors you will hit today

The non-nullable variable 'x' must be assigned before it can be used.

Give it a value, make the type nullable with ?, or mark it late.

LateInitializationError: Field 'accountNumber' has not been initialized.

A late field was read before it was set, which is Task II's case.

The property 'length' can't be unconditionally accessed because the receiver can be null.

Use ?. or check for null first. Do not reach for ! to silence it.

Null check operator used on a null value.

A ! that turned out to be wrong. Replace it with ?? or a real check.

The argument type 'String?' can't be assigned to the parameter type 'String'.

Promote it with an if (x != null) branch, or supply a fallback with ??.

How to hand this in

Everything goes to your GitHub repository. No Word documents, no email, no Teams upload.

Commit your work to a branch named lab2, open a pull request, and merge it once it works.

Screenshots asked for in a task go in a /screenshots folder in the same repository.

Your commit history is part of the assessment. Commit as you go, not once at the end.

This lab asks for no screenshots; the committed code and a running app are the evidence.

Checklist

- code committed and pushed
- screenshots where asked
- app builds and runs
- pull request merged

Wrapping up



Recap

A nullable type is a promise the compiler keeps checking

late defers that promise, but you still have to keep it

?? and ?. replace hand-written defensive null checks

Collections return empty lists, never null



Before next lab

Merge your lab2 pull request

Skim the Flutter widget catalog; Lab 3 builds two screens

Bring the same project: Lab 3 continues inside it



Read more

dart.dev/null-safety

dart.dev/language/classes

dart.dev/language/collections

api.flutter.dev · [ListView.builder](#)